

Azure DevOps for QA Teams

Test Plans • Test Cases • Bugs • Pipelines • Reporting

Brian Scott McCarthy | Senior QA Lead | QA Best Practice Series — BP-01

1. Overview & Philosophy

Azure DevOps (ADO) is the single source of truth for all QA activity. Every test case, every defect, every test run, and every coverage metric lives in ADO — not in spreadsheets, email threads, or local files. This discipline ensures traceability, sprint visibility, and reliable Go/No-Go data.

Core Principle: If it is not in Azure DevOps, it did not happen. Test execution, defect logging, and coverage reporting must be performed inside ADO — not tracked externally.

2. ADO Hierarchy — How QA Organises Work

Understand the ADO hierarchy before creating any artifact:

ADO Level	What It Is	QA Use
Organisation	Top-level account	Shared across all company projects
Project	Your app/product workspace	One per product or product family
Test Plan	Container for a sprint or release cycle	One Test Plan per sprint: TP-[App]-Sprint[##]-[Year]
Test Suite — Static	Manual grouping of test cases by feature area	TS-Authentication-Functional, TS-Cart-Regression
Test Suite — Req. Based	Auto-linked to a User Story	Cases trace directly to acceptance criteria
Test Suite — Query Based	Dynamic; pulls cases by filter (e.g. all P1 cases)	QBS-P1-Active, QBS-Failed-Sprint##
Test Case	One scenario; steps + expected results	TC-[#####]-[Short Description]
Bug	Defect work item; linked to test case + story	BUG-[#####]-[Short Description]

3. Creating a Test Plan

Create one Test Plan per sprint at sprint kickoff — before test case authoring begins.

Field	Value / Guidance
Name	TP-[AppName]-Sprint[##]-[YYYY] e.g. TP-WebApp-Sprint42-2026
Iteration	Select the current sprint iteration path
Area Path	Select your team's area (e.g. WebApp\QA)
Start Date	Sprint start date
End Date	Sprint end date / release date
Owner	Assign to the QA Lead
Description	Brief scope statement: modules in scope, out of scope, environments

- After creating the plan, add Static Suites for each feature area in the sprint.

- Add Requirement-Based Suites by linking to the sprint's User Stories (Board → select US → Add to Suite).
- Add a Query-Based Suite for 'All P1 Active Bugs' so the team always has a live P1 dashboard.

4. Writing Test Cases in ADO

Every test case must be self-contained — a new QA engineer with no context should be able to execute it correctly.

Field	Requirement	Example
Title	TC-[#####]-[Action]-[Context]	TC-0021-PlaceOrder-ValidCreditCard
Area Path	Match your team area	WebApp\QA
Iteration	Assign to authoring sprint	Sprint 42
Priority	1–4 matching severity	1 (P1 — blocking)
Assigned To	Tester responsible for execution	Brian McCarthy
Steps — Action	Clear, atomic action (one thing per step)	Enter email 'testuser@qa.com' in the Email field
Steps — Expected	Observable, measurable outcome	Field accepts input; no validation error displayed
Parameters	Data-driven values for shared steps	@email, @password (linked to parameter table)
Linked Items	Link to US or Feature	Link type: Tests → User Story US-4821
Attachments	Wireframes, test data files, API schemas	checkout_wireframe.png, test_data.csv

5. Executing Tests — Running a Test Suite

Always execute tests through ADO Test Plans — never mark pass/fail in a separate document.

Step	Action in ADO
1	Open Test Plans → select your Test Plan → select the Suite to execute
2	Click 'Run' (top menu) → 'Run for web application' (or 'Run with options' for specific browser)
3	The Test Runner opens alongside the application — step through each action and mark each step Pass / Fail
4	If a step fails → click 'Create Bug' directly from the Test Runner (auto-links to the test case)
5	Add a screenshot or video to the bug before closing the Test Runner step
6	Mark the overall test case as Passed / Failed / Blocked / N/A
7	After execution, refresh the Test Plan to see live pass/fail counts and coverage %

Never mark a test case as Passed if you did not fully execute every step. A partially executed test must be marked Blocked with a comment explaining why.

6. Logging Bugs in ADO

Every defect must be logged in ADO with enough information for a developer to reproduce it without asking a single question.

Bug Field	Requirement
Title	BUG-[####]-[Short Description] — action + context, not vague ('Checkout broken')
Severity / Priority	P1 Critical / P2 High / P3 Medium / P4 Low — set both Severity and Priority fields
Assigned To	Assign to Dev Lead initially; they re-assign to owning developer
Area Path & Iteration	Match the sprint in which it was found
Repro Steps	Numbered, exact steps from a clean state — assume nothing
System Info	Browser, OS, screen resolution, app version/build number, environment (QA/Staging)
Actual Result	Exactly what happened — include error text, console messages
Expected Result	Exactly what should have happened per requirements/acceptance criteria
Attachments	Screenshot, video, console log, HAR file, SQL query output — minimum 1 screenshot
Linked Items	Link to: the failing Test Case (Tested By) + the User Story (Related)
Tags	Add tags: regression, smoke, P1, module name — enables query-based reporting

7. ADO Reporting & Metrics

Use ADO's built-in charts and dashboards to communicate quality status to stakeholders without exporting to spreadsheets.

Report / Chart	Where to Find It	Use It To
Test Plan Progress	Test Plans → Progress tab	Track pass/fail/blocked counts per suite in real time
Defect Trend Chart	Boards → Charts → New Chart → Trend	Show defects opened vs closed per sprint
Test Results Trend	Dashboards → Add Widget → Test Results Trend	Monitor sprint-over-sprint pass rate improvement
Coverage Matrix	Test Plans → select suite → Column options → add 'Outcome'	Show which User Stories have test coverage
Query-Based Bug Board	Boards → Queries → New Query → Type=Bug, State<>Closed	Live view of all open defects with severity breakdown
Pipeline Test Results	Pipelines → select run → Tests tab	Automated test pass/fail with per-test details from CI

Sprint Ritual: Pin a Test Plan Progress chart and a Defect Trend chart to the team dashboard. Review both in every sprint standup so the whole team — not just QA — sees quality status.

8. CI/CD Pipeline Integration

Integration	How to Set Up	Benefit
Publish test results	Add 'Publish Test Results' task in YAML pipeline; point to JUnit/NUnit XML output	Automated test results visible in ADO pipeline run dashboard
Fail build on test failure	Set failTaskOnFailedTests: true in the Publish task	PR blocked from merge if automation suite fails
Link pipeline to Test Plan	In Test Plans → Settings → associate pipeline with plan	Automated runs appear inside the Test Plan alongside manual results
Auto-create bugs from failures	Use ADO REST API or the 'Create Work Item on failure' task	Persistent failures auto-generate Bug work items

9. ADO Quick-Reference Cheat Sheet

Action	Keyboard / Path
Create new Test Plan	Test Plans → + New Test Plan
Add test case to suite	Suite → + New Test Case OR + Add Existing
Run tests	Suite → Run → Run for web application
Log a bug from test run	Test Runner → Fail step → Create Bug
Link test case to User Story	Test Case → Links tab → Add Link → Tests
Create query-based suite	Suite → + New Suite → Query-based Suite
Export test cases to Excel	Suite → Export → Export to CSV
View pipeline test results	Pipelines → [run] → Tests tab
Add chart to dashboard	Dashboards → Edit → Add Widget → Test Results Trend